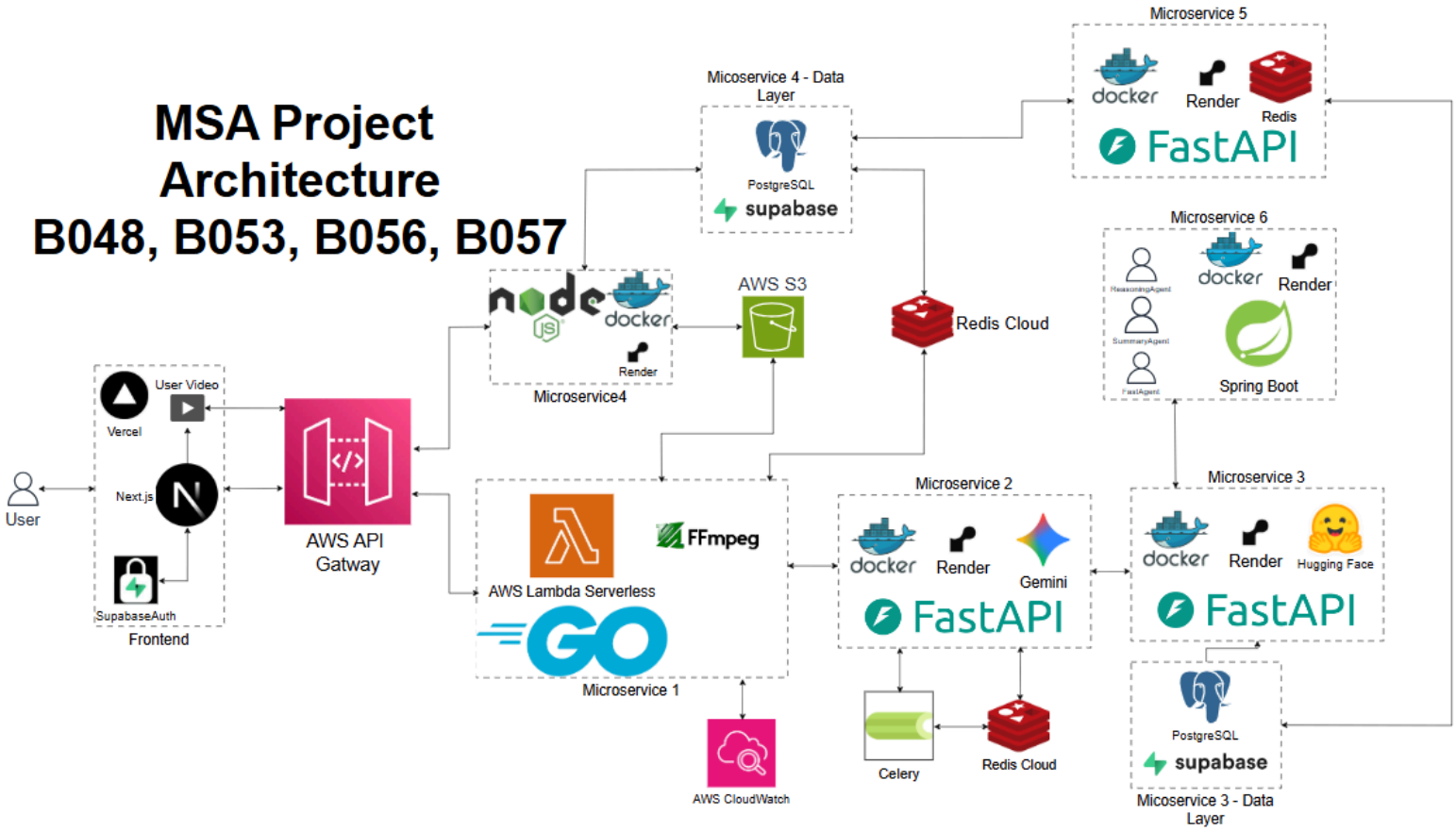
MSA Project - B048, B053, B056, B057

NeuroStream is a high-performance polyglot microservices platform designed to transform raw video uploads into searchable conversational intelligence by leveraging specialized processing engines. The architecture utilizes a Node.js orchestrator (MS4) to manage user workflows, billing, and secure S3 uploads , while a Go-based media processor (MS1) handles high-concurrency video chunking and frame extraction. Intelligence is generated through a Python AI pipeline (MS2) that creates multi-lingual transcripts and visual embeddings , which are then indexed by a FastAPI service (MS3) for sub-second vector search. Finally, the platform provides advanced value through a FastAPI analytics engine (MS5) that tracks per-user behavioral insights and an Agentic Researcher (MS6) that enables complex RAG-based conversational synthesis over indexed content , all supported by a robust data layer featuring PostgreSQL for relational integrity, Redis for asynchronous coordination, and AWS S3 for object storage.

1) Detailed Architecture Diagram



2) List out all the functionality

The platform is divided into functional domains that handle the lifecycle of a video from ingestion to conversational analysis.

- **User Workflow Management (MS4 - Node.js)**
    - Handles user authentication and authorization.
    - Generates secure S3 presigned URLs for direct video uploads to bypass server bottlenecks.
    - Registers uploaded videos and maintains metadata in PostgreSQL.
    - Triggers downstream processing by pushing jobs to the Redis queue.
    - Tracks the workflow status of every video across all processing stages.
    - Manages user subscriptions, quotas, and billing records.
    - Supports library operations such as renaming, deleting, and fetching video details.
- **Media Processing (MS1 - Go)**
    - Consumes raw video files from S3 and performs high-concurrency chunking.
    - Wraps FFmpeg to execute transcoding, audio extraction, and frame sampling.
    - Optimizes media for AI consumption by creating normalized segments.
- **AI Vision and NLP Perception (MS2 - FastAPI)**
    - Processes audio segments to generate timestamped multi-lingual transcripts.
    - Analyzes visual frames using Gemini 3 Flash to identify objects and on-screen text.
    - Converts textual and visual data into high-dimensional vector embeddings.
- **Search and Discovery (MS3 - FastAPI)**
    - Indexes transcript text and vector embeddings into a searchable database.

    - Performs low-latency vector similarity searches to locate specific content within videos.
    - Manages complex metadata queries and searchable readiness status.
- **Personalized Analytics (MS5 - FastAPI)**
    - Tracks per-user behavior including frequently searched timestamps and revisited segments.
    - Identifies important moments to generate smart highlights and personalized insights.
- **Agentic Researcher and RAG (MS6 - Springboot)**
    - Orchestrates multi-step Retrieval-Augmented Generation (RAG) workflows.
    - Synthesizes conversational responses by reasoning across multiple video segments

3) Define scale/ Input -Outputs

Service	Primary Inputs	Primary Outputs
User Workflow (MS4)	Filename, content type, upload completion notification	Presigned URL, current workflow status, searchable readiness status
Media Processor (MS1)	Storage path, processing job from Redis	Status updated, media processing completion status
AI Perception (MS2)	Media chunks, visual frames	Transcript and frame-analysis completion status
Search/Librarian (MS3)	Transcripts, vectors, metadata references	Searchable/index-ready updates

Video Analytics (MS5)	User interaction events, query history, video ID	Important sections, smart highlights, analytics completion status
The Brain (MS6)	User questions, indexed video content	Advanced conversational responses, synthesized research

4) Mark Microservices and their interaction

- **Ingestion Request:** The user frontend requests an upload URL from MS4 through the AWS API Gateway.
- **Storage:** The frontend uploads the video file directly to S3.
- **Pipeline Trigger:** MS4 registers the video in PostgreSQL and pushes a processing job into the Redis queue.
- **Sequential Processing:**
- MS1 consumes the job, processes the media, and updates its status.
- MS2 receives the processed chunks to perform transcription and vision analysis.
- MS3 receives the results from MS2 to index the data for search.
- **Completion Callbacks:** Each service sends status updates back to MS4 (e.g., media processing completion, AI completion, indexing completion) .
- **User Access:** MS4 updates the final state in the user library, making the video available for search and analysis.
- **Conversational Loop:** When a user asks a question, MS4 routes the request to MS6, which queries MS3 for context before returning a synthesized answer.

5) Database selection

PostgreSQL (Relational & Vector Data Layer)

PostgreSQL serves as the robust Source of Truth and relational backbone for the platform. It is utilized in Microservice 4 to manage highly structured

data like user accounts, billing records, and video metadata with strict ACID compliance. Additionally, in Microservice 3, the pgvector extension is leveraged to store high-dimensional embeddings generated by Gemini and Hugging Face models. This allows for seamless Hybrid Search by querying structured metadata like video title or date and unstructured semantic data such as the transcript meaning within a single, consistent database environment.

Redis (Asynchronous Message & State Layer)

Redis acts as the high-speed nervous system that coordinates communication across the polyglot stack. It is primarily used for asynchronous job coordination and workflow queue management. When an upload completes in the Node.js service, a job is pushed to Redis; the Go and Python workers then consume these tasks without blocking the main user experience. In Microservice 2, it specifically supports Celery to manage long-running AI transcription tasks, ensuring that the system can handle bursts of traffic by scaling workers horizontally while maintaining low-latency state access.

AWS S3 (Object Storage Layer)

AWS S3 is the specialized Giant Vault designed for large-scale binary objects (BLOBs) that are too heavy for traditional databases. It stores raw video uploads, audio chunks, and extracted keyframes, keeping the database lean and performant. By utilizing S3 Presigned URLs, the architecture allows the frontend to upload large video files directly to the cloud, bypassing the application servers to save significant bandwidth and prevent processing bottlenecks. It provides the durability and high availability required for global video delivery and long-term asset archival.

Rate limits and Project scale

AWS S3

- No concurrency limits or tier throttling.
- Automatically scales to handle essentially limitless concurrent requests.
- 5 GB of Standard storage per month.

AWS Lambda

- Free and paid tiers share the same limits.
- 1,000 concurrent executions per region by default (can be increased via support).
- Handles sudden traffic spikes with bursts of 500 to 3,000 concurrent executions.
- 1 million free requests per month.

Render

- Free Tier: Strictly limited to 100 concurrent connections. Instances spin down after 15 minutes of inactivity, causing cold starts for new requests.
- Hardcoded limits based on the specific plan, ranging from 100 connections on Basic/Pro-4GB instances up to a maximum of 500 connections on 32GB+ instances.